

Documentation technique

Data Lake financier

Groupe : Artemiy Smogunov, Nathan Smadja-Tubiana, Patrice Ignongui

Projet EFREI 2025-2026

Data Lake financier

Projet final du cours Data Lakes & Data Integration, EFREI 2025-2026.

Auteurs :

- Artemiy Smogunov
- Nathan Smadja-Tubiana
- Patrice Ignongui

Le sujet est disponible dans [consignes/Data_Lakes_Projet_Final_EFREI_2025-2026.pdf](#).

Fonctionnement du projet

Le projet collecte des cours financiers journaliers depuis deux sources :

- le fichier versionné `data/finance_dataset.csv` ;
- Yahoo Finance, interrogé avec la bibliothèque `yfinance`.

Les données passent par trois zones.

```
Fichier CSV -----|
                    |--> Raw --> Staging --> Curated --> FastAPI
Yahoo Finance -----|
                        Airflow orchestre ce flux
```

Raw conserve les données reçues dans MinIO et les indexe dans Elasticsearch. Staging nettoie les lignes et calcule les indicateurs techniques dans PostgreSQL. Curated détecte les anomalies, calcule une tendance et produit un signal simplifié dans PostgreSQL.

Services lancés par Docker Compose

- PostgreSQL, port 5432 : tables Staging, Curated, journaux d'ingestion et base Airflow.
- MinIO, ports 9000 et 9001 : objets CSV et JSON de la zone Raw.
- Elasticsearch, port 9200 : documents Raw interrogeables.
- Airflow, port 8080 : planification et suivi du DAG.
- FastAPI, port 8000 : accès aux données et déclenchement manuel des pipelines.

Lancement

Prérequis : Docker avec Docker Compose.

```
git clone https://github.com/Nathan2412/Projet-DataLake-Final.git
cd Projet-DataLake-Final
docker compose up -d
```

Le premier démarrage crée les tables PostgreSQL, les deux buckets MinIO et l'utilisateur Airflow.

Vérification :

```
docker compose ps
curl http://localhost:8000/health
```

Interfaces :

- documentation FastAPI : <http://localhost:8000/docs>
- Airflow : <http://localhost:8080>
- MinIO : <http://localhost:9001>

Les identifiants de l'environnement local sont définis dans `docker-compose.yml`.

Pipeline Airflow

Le DAG s'appelle `financial_data_lake_pipeline`. Il est planifié à 6 h UTC du lundi au vendredi.

Ordre d'exécution :

```
start
|-- ingest_file --|
|                  |--> transform_staging --> transform_curated --> log_summary --> end
|-- ingest_api  --|
```

`ingest_file` et `ingest_api` s'exécutent en parallèle. Staging commence après la fin des deux ingestions. Curated commence après Staging.


17:24 UTC
AU

[DAGs](#)
[Cluster Activity](#)
[Datasets](#)
[Security](#)
[Browse](#)
[Admin](#)
[Docs](#)

DAGs

All 1
Active 1
Paused 0
Running 0
Failed 0

Auto-refresh

DAG	Owner	Runs	Schedule	Last Run	Next Run	Recent Tasks	Actions
 financial_data_lake_pipeline data-lake finance yfinance	data-engineer	3 0 0	0 6 * * 1-5	2026-07-10, 17:17:03	2026-07-10, 06:00:00	7 0 0 0 0 0 0	 

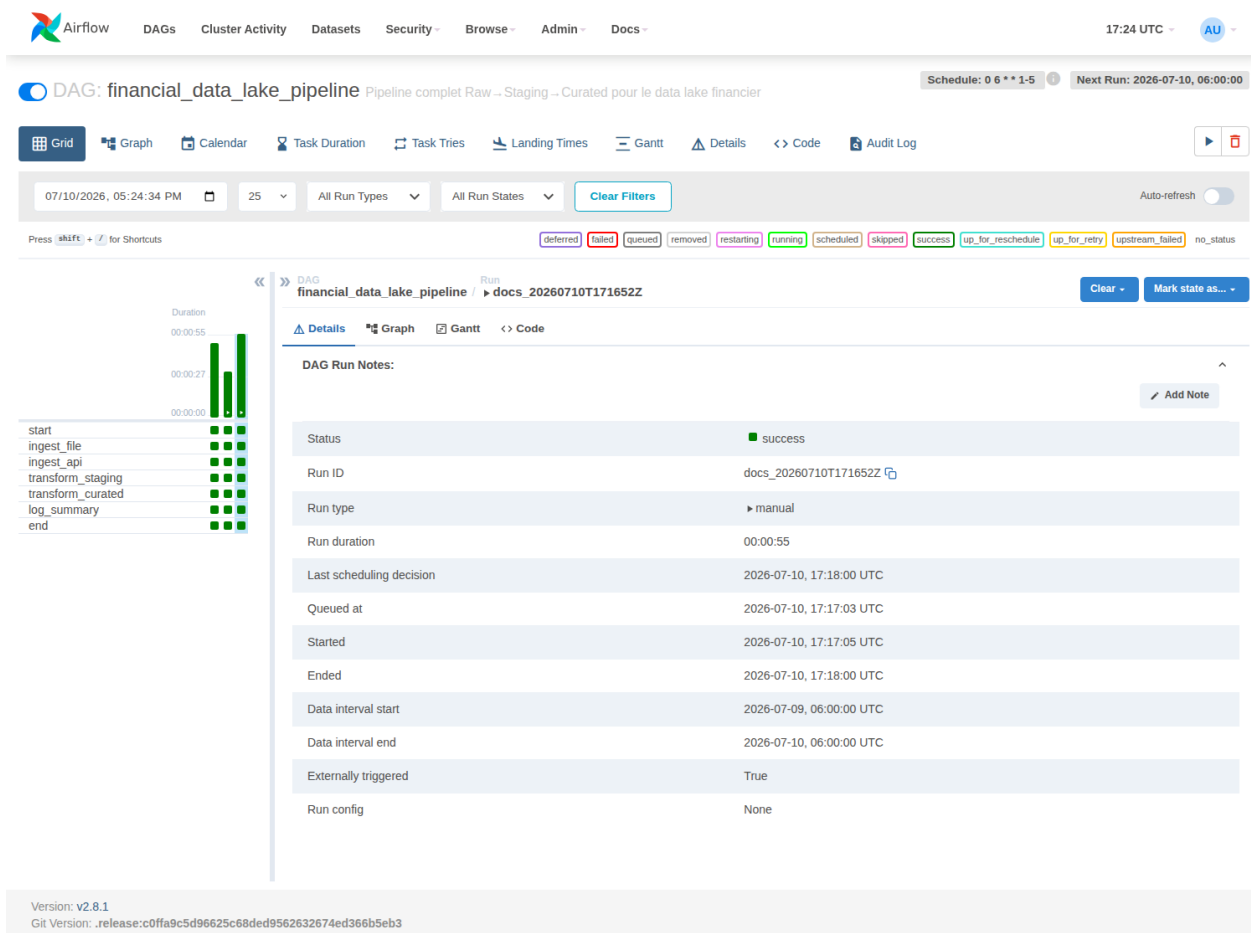
«
<
1
>
»

Showing 1-1 of 1 DAGs

Version: v2.8.1
Git Version: .release:c0ffa9c5d96625c68ded9562632674ed366b5eb3

Liste des DAGs dans Airflow

La capture montre le seul DAG du projet, son planning 0 6 * * 1-5 et sept tâches récentes terminées avec succès.



Exécution du DAG

Cette exécution manuelle s'est terminée en 55 secondes. Les tâches `start`, `ingest_file`, `ingest_api`, `transform_staging`, `transform_curated`, `log_summary` et `end` sont toutes en succès.

API

La page Swagger regroupe les routes utilisées par le projet.

Financial Data Lake API 1.0.0 OAS 3.1

/openapi.json

API Gateway du data lake financier (Yahoo Finance / yfinance). Expose les trois zones du data lake (Raw, Staging, Curated) et fournit des endpoints d'ingestion standard et optimisé.

Health		^
GET	/health	Vérification de l'état des services
Stats		^
GET	/stats	Métriques de remplissage du data lake
Raw		^
GET	/raw	Données brutes de la zone Raw
GET	/raw/objects	Liste des objets MinIO dans la zone Raw
Staging		^
GET	/staging	Données staging avec indicateurs techniques
GET	/staging/tickers	Liste des tickers disponibles en staging
Curated		^
GET	/curated	Données curated avec scores d'anomalie
GET	/curated/anomalies/summary	Résumé des anomalies par ticker
GET	/curated/signals	Tickers avec signaux actifs (buy/sell)
Ingest		^
POST	/ingest	Ingestion synchrone avec benchmark
Ingest Fast		^
POST	/ingest_fast	Ingestion optimisée
Schemas		^
HTTPValidationError > Expand all object		
HealthResponse > Expand all object		
IngestFastRequest > Expand all object		
IngestFastResponse > Expand all object		
IngestRequest > Expand all object		
IngestResponse > Expand all object		
ServiceStatus > Expand all object		
ValidationError > Expand all object		

Documentation Swagger de l'API

Routes disponibles :

- GET /health vérifie PostgreSQL, MinIO et Elasticsearch.
- GET /stats compte les objets Raw, les documents Elasticsearch et les lignes PostgreSQL.
- GET /raw lit les documents Raw dans Elasticsearch.
- GET /raw/objects liste les objets MinIO.
- GET /staging lit les cours nettoyés et les indicateurs techniques.
- GET /staging/tickers donne les tickers et leurs plages de dates.
- GET /curated lit les anomalies, tendances et signaux.
- GET /curated/anomalies/summary regroupe les anomalies par ticker et par type.
- GET /curated/signals retourne les derniers signaux buy ou sell.
- POST /ingest exécute le pipeline séquentiel.
- POST /ingest_fast télécharge et écrit les données par lots parallèles.

Exemple de lecture :

```
curl "http://localhost:8000/raw?ticker=AAPL&limit=3"
curl "http://localhost:8000/staging?ticker=AAPL&limit=3"
curl "http://localhost:8000/curated?ticker=AAPL&limit=3"
```

Exemple d'ingestion manuelle :

```
curl -X POST http://localhost:8000/ingest_fast \
-H "Content-Type: application/json" \
-d '{
  "data": {
    "tickers": ["AAPL", "MSFT", "NVDA"],
    "period": "5d",
    "run_staging": true,
    "run_curated": true
  }
}'
```

Passage dans les trois zones

Passage des données dans les trois zones

Exemple : AAPL, dernière date disponible.

Raw		Staging		Curated	
Date	2026-07-10	Date	2026-07-10	Date	2026-07-10
Clôture	314.5398864746094	Clôture	314.539886	Score anomalie	-0.527124
Volume	14262307	SMA 20	313.493982	Anomalie	False
Source	yfinance_fast	MACD	0.334471	Tendance	neutral
Date d'ingestion	2026-07-10T17:09:55.683990+00:00	Variation	-0.005313	Signal	hold
		Volatilité	0.008536	Traitement	2026-07-10 17:10:06.641214

Raw conserve la donnée reçue et sa provenance.

Staging ajoute les indicateurs techniques.

Curated produit un score d'anomalie, une tendance et un signal.

Exemple AAPL dans Raw, Staging et Curated

La capture suit la dernière ligne AAPL disponible au moment du test.

- Raw contient le cours reçu, le volume, la source et la date d'ingestion.
- Staging contient le même cours après normalisation, avec la moyenne mobile, le MACD, le rendement journalier et la volatilité.

- Curated ajoute le score d'anomalie, le type de tendance et le signal.

Pour cette ligne, `is_anomaly` vaut `false`, la tendance est `neutral` et le signal est `hold`. Le pipeline n'a donc pas classé ce point comme anormal et n'a produit ni signal d'achat ni signal de vente.

Résultat de l'exécution contrôlée

Le 10 juillet 2026, nous avons lancé la stack puis déclenché le DAG `financial_data_lake_pipeline` avec l'identifiant `docs_20260710T171652Z`.

Résultat observé :

- état général de l'API : ok ;
- PostgreSQL : ok ;
- MinIO : ok ;
- Elasticsearch 8.11.0 : ok ;
- 242 objets dans MinIO, dont 200 objets issus du fichier et 42 objets issus de l'API ;
- 1 763 documents dans Elasticsearch ;
- 1 243 lignes dans Staging ;
- 1 243 lignes dans Curated ;
- 37 lignes marquées comme anomalies ;
- sept tâches Airflow terminées avec le statut `success`.

Résultats de l'exécution

10 juillet 2026. Données lues après le run Airflow docs_20260710T171652Z.

Etat des services

PostgreSQL ok	MinIO ok	Elasticsearch ok version 8.11.0
-------------------------	--------------------	--

Volumes observés

Objets MinIO 242 200 fichier, 42 API	Documents Raw ES 1763	Lignes Staging 1243
Lignes Curated 1243	Anomalies Curated 37	Tâches Airflow réussies 7 / 7

Lecture

- | Raw conserve les objets et les documents reçus.
- | Staging et Curated contiennent le même nombre de lignes car Curated enrichit chaque ligne Staging.
- | Les 37 anomalies viennent du modèle Isolation Forest exécuté par ticker.

Résumé de l'exécution

Les nombres Staging et Curated sont identiques parce que Curated enrichit chaque ligne Staging au lieu de filtrer le jeu de données. Les 37 anomalies viennent d'Isolation Forest configuré avec une contamination de 5 %. Ce nombre dépend donc du paramètre du modèle et du contenu chargé.

Transformations Staging

Pour chaque ticker, le code lit les documents Elasticsearch, trie les dates, retire les doublons et ignore les lignes sans cours de clôture. Il calcule ensuite :

- SMA sur 20 et 50 périodes ;
- EMA sur 12 et 26 périodes ;
- RSI sur 14 périodes ;
- MACD et ligne de signal ;
- bandes de Bollinger sur 20 périodes ;
- rendement journalier ;
- volatilité glissante sur 20 périodes.

La clé unique (ticker, date) permet de rejouer une ingestion sans créer une nouvelle ligne pour la même date.

Transformations Curated

Isolation Forest utilise quatre variables : rendement journalier, volatilité sur 20 périodes, z-score du volume et RSI. Le modèle ne se lance qu'à partir de 30 lignes pour un ticker. Sa contamination est fixée à 5 %.

Les types d'anomalies sont attribués par des règles visibles dans `transformation/curated/transform_curated.py` :

- baisse journalière inférieure à -5 % : `flash_crash` ;
- hausse supérieure à 5 % : `price_spike` ;
- z-score absolu du volume supérieur à 3 : `volume_spike` ;
- volatilité supérieure à 4 % : `high_volatility` ;
- autre point isolé : `unknown_anomaly`.

La tendance compare le cours, la SMA 20 et la SMA 50. Le signal combine le RSI et le croisement MACD. Il vaut `buy`, `sell` ou `hold`. Ce signal est une sortie technique du projet, pas une recommandation financière.

Comparaison des deux ingestions

`scripts/benchmark_endpoints.py` compare `/ingest` et `/ingest_fast` avec des lots de 1 et 100 tickers. Le mode rapide utilise huit threads pour les téléchargements et les envois MinIO, une écriture groupée dans Elasticsearch et `execute_values` pour PostgreSQL.

Le fichier `livrables/benchmark_ingest_vs_ingest_fast.json` contient trois répétitions par mode et par taille de lot.

Résultat enregistré :

- lot de 1 ticker : 1 073,29 ms contre 989,86 ms, soit 7,77 % de gain ;
- lot de 100 tickers : 104 825,65 ms contre 12 230,36 ms, soit 88,33 % de gain.

Le gain est faible sur un seul ticker. Sur 100 tickers, les opérations réseau parallèles et les écritures groupées réduisent fortement le temps total.

Tests

```
python -m pytest -q
```

Résultat vérifié : 12 tests réussis.

Les tests couvrent les calculs RSI, les tendances, les signaux, la préparation Staging, les identifiants Raw, le chargement du CSV et le calcul du benchmark.

Limites observées

- Yahoo Finance est un service externe. La durée et le nombre de lignes disponibles changent selon sa réponse.
- Le mode rapide calcule les indicateurs Staging sur le lot téléchargé. Avec une période de cinq jours, le RSI 14 reste vide car le lot ne contient pas quatorze observations.
- Isolation Forest ne s'exécute pas sous 30 lignes pour un ticker.
- Les signaux Curated n'ont pas été évalués comme stratégie financière.
- Les identifiants présents dans Docker Compose sont prévus pour cet environnement local de cours.

Documentation

- `livrables/RAPPORT_TECHNIQUE.md` décrit le fonctionnement et les résultats.
- `docs/CONFORMITE_DEVOIR.md` relie le projet aux demandes du sujet.
- `data/README.md` décrit le fichier CSV versionné.
- `docs/captures/` contient les captures utilisées dans la documentation.

Régénération des PDF :

```
.venv/bin/pip install -r scripts/requirements-docs.txt  
.venv/bin/python scripts/generate_pdf_deliverables.py
```

Vérification par rapport au sujet

Sujet : Data Lakes & Data Integration, projet final EFREI 2025-2026.

Cette page relie les demandes du sujet aux fichiers et aux résultats observés le 10 juillet 2026.

Data lake de bout en bout

Le flux exécuté est le suivant :

Deux sources --> Raw --> Staging --> Curated --> API

Preuves dans le dépôt :

- ingestion : ingestion/ingest_file.py et ingestion/ingest_api.py ;
- transformation : transformation/staging/transform_staging.py et transformation/curated/transform_curated.py ;
- stockage : docker-compose.yml et scripts/init_db.sql ;
- orchestration : airflow/dags/financial_pipeline_dag.py ;
- exposition : api/main.py et api/routers/.

Deux sources

Le projet utilise réellement :

- data/finance_dataset.csv comme source fichier versionnée ;
- Yahoo Finance par la bibliothèque yfinance comme source API.

L'exécution contrôlée a produit 200 objets dans le bucket fichier et 42 objets dans le bucket API.

Trois zones

Raw utilise deux stockages :

- MinIO pour les objets CSV et JSON ;
- Elasticsearch pour les lignes brutes interrogeables.

Staging utilise la table PostgreSQL staging_ohlcv.

Curated utilise la table PostgreSQL curated_analysis.

Après l'exécution, Staging et Curated contenaient chacun 1 243 lignes.

Pipeline Airflow

Le DAG `financial_data_lake_pipeline` exécute sept tâches. Le run `docs_20260710T171652Z` s'est terminé en 55 secondes avec sept statuts succès.

La capture se trouve dans `docs/captures/airflow-execution.png`.

Le planning du code est `0 6 * * 1-5`, soit 6 h UTC du lundi au vendredi.

API Gateway

FastAPI expose les routes demandées :

- `/raw` ;
- `/staging` ;
- `/curated` ;
- `/health` ;
- `/stats`.

Le projet ajoute aussi `/raw/objects`, `/staging/tickers`, `/curated/anomalies/summary`, `/curated/signals`, `/ingest` et `/ingest_fast`.

La capture Swagger se trouve dans `docs/captures/api-swagger.png`.

Traitement avancé

La zone Curated applique Isolation Forest à quatre variables et enregistre le score, le booléen d'anomalie et son type. Elle calcule aussi une tendance et un signal.

L'état observé contenait 37 anomalies sur 1 243 lignes Curated.

Ingestion optimisée

`/ingest` traite les téléchargements séquentiellement.

`/ingest_fast` utilise :

- huit threads pour les téléchargements ;
- huit threads pour les envois MinIO ;
- une écriture Elasticsearch groupée ;
- `execute_values` pour l'upsert PostgreSQL.

Le benchmark enregistré compare les deux routes avec trois répétitions.

Pour 100 tickers, la médiane passe de 104 825,65 ms à 12 230,36 ms. Le gain mesuré est 88,33 %.

Les mesures et statuts de chaque appel sont dans `livrables/benchmark_ingest_vs_ingest_fast.json`.

Documentation et captures

Le dépôt contient :

- le guide de lancement dans README.md ;
- le rapport dans livrables/RAPPORT_TECHNIQUE.md ;
- cette vérification ;
- la description du CSV dans data/README.md ;
- cinq captures dans docs/captures/ ;
- deux PDF produits par scripts/generate_pdf_deliverables.py.

Contrôles exécutés

```
docker compose up -d
curl http://localhost:8000/health
curl http://localhost:8000/stats
docker compose exec airflow-scheduler airflow dags trigger financial_data_lake_pipeline
PYTHONPATH=./api .venv/bin/python -m pytest -q
```

Résultats :

- services PostgreSQL, MinIO et Elasticsearch en état ok ;
- run Airflow terminé avec sept tâches en succès ;
- endpoints Raw, Staging et Curated accessibles ;
- 12 tests réussis.